TENNR · IMPLEMENTATION ENGINEER · TAKE-HOME ASSIGNMENT

Wikipedia → Google Sheets

An RPA workflow that extracts **Year, Winner, Score, Runners-up** from the first 10 rows of the *"List of FIFA World Cup finals"* table and appends each row to a Google Sheet via the Google Sheets API (configured exactly as a Postman request) — built **exclusively from the 9 steps in the provided IE Step Library**.

Candidate · Bhunesh Bansal **Date** · 7 July 2026 **Source** · en.wikipedia.org/wiki/List_of_FIFA_World_Cup_finals **Target** · Google Sheet — tab `Sheet1`, columns A:D

A Reading guide — the Step Library

Every executable node below is one of the library's 9 steps (color-coded). Steps the task does not need are accounted for, with the reason.

Open page
starts the RPA browsing session
`used ×1`

Loop
fixed-count iteration, index from 1
`used ×1 (10 iterations)`

ExtractHTML
DOM path in → element text out
`used ×4 per iteration`

Parse number
numeric string → Int
`used ×1 per iteration`

Detour
"if" — run branch when true, then resume
`used ×1 per iteration`

Call API
configured exactly like Postman
`used ×1 per iteration (+1 optional QA)`

Fill text field
types into an input
`not needed – read-only page, nothing to type`

Click
simulated mouse click
`deliberately avoided – clicking a column header would re-sort rows and scramble tr[index]`

Check Box
toggles a checkbox
`not needed – page has no checkboxes`

B One-time setup (before the workflow runs)

Preconditions only — these are **not** workflow steps and use nothing from the Step Library.

PREPARED ONCE, REUSED ON EVERY RUN

S1 Target spreadsheet
Create a Google Sheet (or convert the uploaded Excel file to Google Sheets format — the API can only write to native Sheets). Tab `Sheet1`; headers `Year | Winner | Score | Runners-up` in A1:D1. Copy the **spreadsheetId** from the URL between `/d/` and `/edit`.

S2 Google Cloud
Create a project → enable the **Google Sheets API** → create OAuth 2.0 client credentials (the sheet's owner account must grant access).

S3 Postman auth
OAuth 2.0 (Authorization Code) → obtain `{{ACCESS_TOKEN}}` with scope `…/auth/spreadsheets`. Full config in panel D.

S4 Capture DOM paths
Per the resource video: DevTools → Inspect element → right-click the node → **Copy** → **Copy XPath**. Replace the row number with `{{index}}`. Paths verified in panel E.

C The workflow — flowchart

One straight spine: open the page, loop over the 10 finals rows, and inside every iteration extract 4 fields, type-check the year, guard with a Detour, and append the row via one API call. Loop index `i` does triple duty: iteration counter ↔ table row `tbody/tr[i]` ↔ sheet row `i+1`.

START

1 **Open the source page** — `Open page`

Begins the RPA browsing session (per library note) and loads the article that hosts the target table.

INPUT `https://en.wikipedia.org/wiki/List_of_FIFA_World_Cup_finals`
Page URL — the only required input.

OUTPUT Live browser session with the page fully rendered. Once the sortable table initialises, its header row moves into **<thead>**, so data rows are **tbody/tr[1] … tbody/tr[22]** — row 1 = the 1930 final.

Keep default (chronological) sort. No clicks on column headers — sorting re-orders `tr` indices and would corrupt the extraction. The table renders on page load; no interaction is needed.

page_ready

2 Loop — once per finals row

Loop

Input: number of iterations = **10** · **Output:** index `i` = 1, 2, … 10 (library: index starts from 1)

i = 1 → 1930 … i = 10 → 1974

error inside an iteration ⇒ run continues at next index (library note)

rows are processed top-to-bottom, so sheet order = table order

2.1 Extract “Year”

ExtractHTML

INPUT

```
//*[@id="mw-content-text"]/div[2]/section[2]/table[3]/tbody/tr[{{index}}]/th/a
```

Element DOM path (XPath) — year link inside the row-header cell.

OUTPUT Text → `year_text` e.g. i=1 → **"1930"**

2.2 Convert year to a number

Parse number

INPUT

```
year_text
```

String, e.g. **"1930"**

OUTPUT

```
year
```

Int, e.g. **1930** — lands in column A as a true number, not text.

Free row-guard: if markup ever shifts and a non-numeric string is extracted, this step errors → the Loop skips to the next index (library behaviour). Bad rows can never reach the sheet.

2.3 Extract “Winner”

ExtractHTML

INPUT

```
//*[@id="mw-content-text"]/div[2]/section[2]/table[3]/tbody/tr[{{index}}]/td[1]/a
```

Anchor text = clean country name (skips the flag-icon span in the same cell).

OUTPUT Text → `winner` e.g. i=1 → **"Uruguay"**

2.4 Extract “Score”

ExtractHTML

INPUT

```
//*[@id="mw-content-text"]/div[2]/section[2]/table[3]/tbody/tr[{{index}}]/td[2]
```

Whole cell — keeps the “(a.e.t.)” qualifier where a final went to extra time (1934, 1966).

OUTPUT Text → `score` e.g. i=1 → **"4–2"**, i=2 → **"2–1 (a.e.t.)"**

Scores use an **en dash (–, U+2013)**, not a hyphen. Row 4 (1950) carries a footnote marker in the same cell — see edge case **#3**.

2.5 Extract “Runners-up”

ExtractHTML

INPUT

```
//*[@id="mw-content-text"]/div[2]/section[2]/table[3]/tbody/tr[{{index}}]/td[3]/a
```

Anchor text = clean country name.

OUTPUT Text → `runners_up` e.g. i=1 → **"Argentina"**

2.6 ◇ Append guard — is the row complete?

Detour

Library: “conditional execution like an ‘if’ statement; if true, run branch then resume the run.” One branch, one condition.

INPUT

```
winner ≠ "" AND score ≠ "" AND runners_up ≠ ""
```

`year` is already validated by step 2.2 — this guards the three text fields against a silent partial extraction.

OUTPUT true → branch executes (2.7), then the run resumes · false → branch skipped, loop moves on

next index · i ≤ 10 · repeat ↺

condition FALSE

Branch does not run. Nothing is appended — **no partial rows ever reach the sheet**.

Run resumes → next index.

condition TRUE → run branch

2.7 Append this row to Google Sheets

Call API

One POST per row, configured exactly as a Postman request — full spec in panel D.

INPUT

`POST …/values/Sheet1!A:D:append`

Body → `"values": [[ {{year}}, "{{winner}}", "{{score}}", "{{runners_up}}" ]]`

e.g. i=1 → **[[1930, "Uruguay", "4–2", "Argentina"]]**

OUTPUT

HTTP 200 · `updates.updatedRange = "Sheet1!A{{index+1}}:D{{index+1}}"` (i=1 → A2:D2)

If the call fails (401/429/network), the iteration errors and the Loop **skips to the next index** — one bad row never blocks the other nine.

both paths converge → run resumes → Loop advances to next index

after iteration 10 → exit loop ▼

all rows processed

3 Verify the write (optional QA — recommended)

Call API

INPUT

```
GET
https://sheets.googleapis.com/v4/spreadsheets/{{SPREADSHEET_ID}}/values/Sheet1!A:D
```

OUTPUT

200 with `values` array of **11 rows** (1 header + 10 data) — confirms every append landed.

END

Final state: Sheet1!A2:D11 = the first 10 FIFA World Cup finals, 1930 → 1974 (full data in panel F)

D The API call — Google Sheets `values:append` via Postman

Exact configuration for step 2.7. Everything in `{{double_braces}}` is a Postman/workflow variable.

POST `https://sheets.googleapis.com/v4/spreadsheets/{{SPREADSHEET_ID}}/values/{{RANGE}}:append`

PATH VARIABLES

VARIABLE	VALUE / WHERE IT COMES FROM
`SPREADSHEET_ID`	From the sheet URL: `docs.google.com/spreadsheets/d/<this>/edit`
`RANGE`	`Sheet1!A:D` — the table to append to (Postman URL-encodes `!` and `:` automatically)

QUERY PARAMETERS

PARAM	WHY
`valueInputOption=RAW`	Write values literally. `USER_ENTERED` would let Sheets re-interpret strings (dash-separated scores can coerce to dates); RAW is deterministic.
`insertDataOption=INSERT_ROWS`	Always add new rows below the existing table — never overwrite.
`includeValuesInResponse=true`	Optional: response echoes what was written — instant per-row verification.

HEADERS

`Content-Type`	`application/json`
`Authorization`	`Bearer {{ACCESS_TOKEN}}` (set automatically by Postman's OAuth 2.0 helper)

AUTH — POSTMAN OAUTH 2.0 (AUTHORIZATION CODE)

Auth URL	`https://accounts.google.com/o/oauth2/v2/auth`
Token URL	`https://oauth2.googleapis.com/token`
Scope	`https://www.googleapis.com/auth/spreadsheets`
Callback	`https://oauth.pstmn.io/v1/callback`

REQUEST BODY — TEMPLATE (ITERATION I)

```
{
  "range": "Sheet1!A:D",
  "majorDimension": "ROWS",
  "values": [
    [ {{year}}, "{{winner}}", "{{score}}", "{{runners_up}}" ]
  ]
}
```

CONCRETE EXAMPLE — ITERATION 1

```
{
  "range": "Sheet1!A:D",
  "majorDimension": "ROWS",
  "values": [
    [ 1930, "Uruguay", "4–2", "Argentina" ]
  ]
}
```

SUCCESS RESPONSE — 200 OK (ITERATION 1)

```
{
  "spreadsheetId": "{{SPREADSHEET_ID}}",
  "tableRange": "Sheet1!A1:D1",
  "updates": {
    "updatedRange": "Sheet1!A2:D2",
    "updatedRows": 1,
    "updatedColumns": 4,
    "updatedCells": 4
  }
}
```

FAILURE MODES

`401`	Token expired → refresh in Postman (Get New Access Token).
`403`	Sheets API not enabled, or account lacks edit access to the sheet.
`404`	Wrong `SPREADSHEET_ID`.
`429`	Quota (60 write req/min/user). 10 sequential appends sit far below it.

Inside the Loop, any failure only skips that row (library note) — remaining iterations still append. `:append` is **not idempotent**: re-running the workflow duplicates rows, so clear A2:D11 before a re-run.

E DOM paths — captured & verified

Method from the resource video: Inspect the element → right-click it in the Elements panel → **Copy** → **Copy XPath** → replace the row number with `{{index}}`. All four paths verified against the live page on 7 July 2026.

FIELD	XPATH TEMPLATE (INPUT TO EXTRACTHTML)	OUTPUT @ I = 1
Year	`//*[@id="mw-content-text"]/div[2]/section[2]/table[3]/tbody/tr[{{index}}]/th/a`	`"1930"`
Winner	`//*[@id="mw-content-text"]/div[2]/section[2]/table[3]/tbody/tr[{{index}}]/td[1]/a`	`"Uruguay"`
Score	`//*[@id="mw-content-text"]/div[2]/section[2]/table[3]/tbody/tr[{{index}}]/td[2]`	`"4–2"`
Runners-up	`//*[@id="mw-content-text"]/div[2]/section[2]/table[3]/tbody/tr[{{index}}]/td[3]/a`	`"Argentina"`

Prefix drift, handled: the resource video (recorded May 2024) copies `//*[@id="mw-content-text"]/div[1]/table[4]/…`. Wikipedia has since moved to section-wrapped markup, so the same click today yields the prefix above. Only the prefix drifts — the row/cell tail `tbody/tr[{{index}}]/th|td[n]` is stable. Re-copy the prefix at build time exactly as the video shows. **Row indexing:** in a live session the sortable table moves its header row into `<thead>` (visible in the video), so `tbody/tr[i]` is the i-th *data* row; in raw no-JS HTML it would be `tr[i+1]`.

F Expected result — the sheet after the run

Real values from the live table (not placeholders). The first 10 rows span 1930–1974: Wikipedia lists no rows for 1942/1946 (tournaments cancelled, WWII), so a fixed loop bound of 10 is exact.

	A	B	C	D
1	**Year**	**Winner**	**Score**	**Runners-up**
2	1930	Uruguay	4–2	Argentina
3	1934	Italy	2–1 (a.e.t.)	Czechoslovakia
4	1938	Italy	4–2	Hungary
5	1950	Uruguay	2–1 [n 3]	Brazil
6	1954	West Germany	3–2	Hungary
7	1958	Brazil	5–2	Sweden
8	1962	Brazil	3–1	Czechoslovakia
9	1966	England	4–2 (a.e.t.)	West Germany
10	1970	Brazil	4–1	Italy
11	1974	West Germany	2–1	Netherlands

Row 5's score keeps Wikipedia's footnote marker **[n 3]** (the 1950 "final" was the deciding match of a round-robin group) — see edge case #3 for the trade-off and the cleaner alternative.

G Edge cases & design decisions

#1 "First 10 rows" ≠ "first 10 World Cups by year"

1942 and 1946 were cancelled and have no table rows, so rows 1–10 cleanly map to 1930–1974. The loop bound of 10 needs no gap-handling.

#2 Append per row, not one batch call

The library has no accumulator step to build a 10-row array across iterations, and per-row appends pair perfectly with the Loop's skip-on-error rule: one bad row costs one row, not the whole write. 10 sequential calls sit far under the 60 write-req/min quota.

#3 The 1950 footnote marker

Row 4's score cell embeds a superscript note → cell text is `2–1[n 3]`. Chosen: extract the whole cell (keeps "(a.e.t.)" on rows 2 & 8) and accept the marker. Alternative: point ExtractHTML at `…/td[2]/a` for a pure `2–1` — at the cost of dropping "(a.e.t.)".

#4 Scores contain an en dash

`4–2` uses U+2013, not a hyphen. With `valueInputOption=RAW` the string is stored verbatim; no risk of Sheets coercing a dash-separated pair into a date.

#5 XPaths drift; the method doesn't

Wikipedia's wrapper markup changed between the resource video (May 2024: `div[1]/table[4]`) and today (`div[2]/section[2]/table[3]`). The workflow stores the prefix as one variable so a future drift is a one-field fix; the `tr[{{index}}]` tail is stable.

#6 Never sort before extracting

The table is user-sortable; clicking a header re-orders `<tr>` elements and would silently remap every index. The workflow performs zero clicks — extraction happens on the default chronological order.

#7 Year as a real number

Parse number converts `"1930"` → `1930` so column A is sortable/filterable as numeric in Sheets — and doubles as a validity check on every row (parse failure ⇒ loop skips that index).

#8 Re-runs are not idempotent

`:append` always adds rows; running the workflow twice writes 20 rows. Clear `Sheet1!A2:D11` before a re-run (or point `RANGE` at a scratch tab while testing).

Verification. Every XPath was round-trip-tested against the live page HTML and all 10 expected rows extracted programmatically on 7 Jul 2026; API shapes match the official `spreadsheets.values.append` reference.

Bhunesh Bansal
Implementation Engineer take-home · Tennr